Pydantic

In [2]:
```
# Pydantic Workshop: Student Model Example

## Import Modules after pip install pydantic
# pip install pydantic in vscode terminal/ !pip install pydantic in jupyter
# Importing necessary modules from pydantic and other standard libraries
from pydantic import BaseModel, Field, field_validator, \
model_validator, AnyUrl, EmailStr, computed_field
from typing import Optional, List, Dict, Any, Deque, Annotated

#Ignoring warnings for cleaner output
import warnings
warnings.filterwarnings("ignore")
```

Defining the Student model using Pydantic's BaseModel

In [6]:
```
class Student(BaseModel):
 # Using Annotated to add validation and metadata to each field
 name : Annotated[str, Field(Optional = False,
                             description="Give the name of the student (Char
                             max_length=50)]
 class_in : Annotated[str, Field(Optional = False,
                                 description="Give the class of the student (
                             max_length=20)]
 age: Annotated[int, Field(Optional = False,
                           description="Give the age of the student (Integer
                           gt=0, le=50)]

 stream : Annotated[str, Field(Optional = False,
                             description="Give the stream of the student (Ch
                             max_length=20)]

 Subjects : Annotated[Dict[str, str], Field(Optional = True,default=None,
                             description="Give the subjects of the student (
                              max_items=3)]

 email: Annotated[EmailStr, Field(Optional = True, default=None,
                             description="Give the email of the student (Val

 last_class_number : Annotated[float, Field(Optional = False,
                             description="Give the last class number of the
                             gt=0, le=100)]
 studying_hours : float

 # Using field_validator to ensure the name is in proper case (title case)
 # This decorator indicates that the validation should be
 # applied after the initial validation of the field
 @field_validator('name', mode='after')

#This is class method
 @classmethod
#Define the method to convert the name to proper case
 def Proper_case(cls, value):
  return value.title()


 # Computed field to calculate the study index
 #based on studying hours and last class number
 @computed_field
 @property

 def study_index(self)-> float:
  return self.studying_hours / self.last_class_number

#model_validator to ensure that the stream is one of the allowed values
 @model_validator(mode='after')

 def subjects_validation(cls, model):
   scn=['bio', 'maths', 'physics', 'chemistry', 'english']
   arts=['history', 'geography', 'political science', 'english']
   commerce=['accountancy', 'business studies', 'economics', 'english']
   if model.stream.lower() == 'science':
     for subject in model.Subjects.keys():
       if subject.lower() not in scn:
         raise ValueError(f"Invalid subject '{subject}' for stream 'Science'
   elif model.stream.lower() == 'arts':
     for subject in model.Subjects.keys():
```

```
        if subject.lower() not in arts:
          raise ValueError(f"Invalid subject '{subject}' for stream 'Arts'. A
    elif model.stream.lower() == 'commerce':
      for subject in model.Subjects.keys():
        if subject.lower() not in commerce:
          raise ValueError(f"Invalid subject '{subject}' for stream 'Commerce
    else:
      raise ValueError("Stream must be one of the following: Science, Arts, C
```

Example usage

In [7]:

```
student_data = {
    "name": "Rakesh Roshan",
    "class_in": "10th",
    "age": 16,
    "stream": "science",
    "Subjects": {"Maths": "A", "Physics": "B", "Chemistry": "A"},
    "email": "john.doe@example.com",
    "last_class_number": 85.5,
    "studying_hours": 12.5
}


# Creating an instance of the Student model using the provided data
student = Student(**student_data)

# Function to print the student information in a formatted manner
def Print_student_info(student: Student):
    print(f"Name: {student.name}")
    print(f"Class: {student.class_in}")
    print(f"Age: {student.age}")
    print(f"Stream: {student.stream}")
    print(f"Subjects: {student.Subjects}")
    print(f"Email: {student.email}")
    print(f"Last Class Number: {student.last_class_number}")
    print(f"Studying Hours: {student.studying_hours}")
    print(f"Study Index: {student.study_index}")

# Print_student_info(student)

temp= student.model_dump()

print(temp)
```

```
{'name': 'Rakesh Roshan', 'class_in': '10th', 'age': 16, 'stream': 'scienc
e', 'Subjects': {'Maths': 'A', 'Physics': 'B', 'Chemistry': 'A'}, 'email':
'john.doe@example.com', 'last_class_number': 85.5, 'studying_hours': 12.5,
'study_index': 0.14619883040935672}
```

In [ ]: